

Technical Optimization Report

Frontend, Server-Side, and Infrastructure Enhancements

1. Frontend Optimization

Use: Next.js server components, route-based code splitting, lazy loading, CDN for assets, and selective hydration.

Why: This lowers initial bundle size, reduces browser-side CPU usage, improves time-to-first-byte and time-to-interactive, and makes dashboard and media-heavy screens more responsive. Avoid: Heavy client-side rendering for entire pages, unnecessary global state, large unoptimized images, and fetching large payloads directly on the browser.

2. Server-Side Optimization

Use: Server-only data access modules, server-side aggregation, batched fetches, field-level selects, and page-level cache controls. Why: This removes redundant request fan-out, reduces database load, shortens server response time, and makes page rendering more deterministic under traffic spikes. Avoid: One database call per object in a loop, full-row fetches for list screens, and unbounded data serialization in request handlers.

3. Supabase Optimization

Use: A dedicated server-side Supabase data layer with typed queries, filters, indexes, pagination, and compact SELECT statements. Why: It limits read cost, reduces latency, and keeps real-time user data queries stable as the dataset grows. Avoid: Unindexed queries, repeated list fetches without pagination, and broad SELECT * calls for large tables.

4. Caching Strategy

Use: Redis or app-level cache for hot read data such as categories, public profile summaries, lookup tables, and repeated query results. Use cache invalidation for stale data and no-store for user-specific, highly dynamic data. Why: Caching reduces repeated database hits and lowers latency during traffic bursts. Avoid: Caching everything indiscriminately, caching sensitive data without invalidation, and keeping stale user state on the browser for too long.

5. VideoSDK / External API Optimization

Use: Request timeouts, retry policies, exponential backoff, per-meeting cache, deduplication of recording URL checks, and centralized error handling for all external API calls. Why: VideoSDK calls are latency-sensitive and can fail or return partial data due to async processing. Timeouts and caching stabilize UX and reduce API pressure. Avoid: Sequential polling loops, repeated external API calls without cache, and blocking request handling on long-running recording lookups.

6. Docker and Containerization

Use: Dockerized deployment, health checks, non-root runtime users, separate build and runtime stages, and environment-based configuration. Why: It creates reproducible environments and reduces deployment drift

between local, staging, and production. Avoid: Hardcoding secrets into the image, running the app as root, and relying on local machine state for runtime behavior.

7. Kubernetes and Orchestration

Use: Kubernetes Deployment, Service, Ingress, autoscaling, readiness/liveness probes, resource requests, and managed secrets. Why: It enables replication, self-healing, and autoscaling for traffic spikes and system failures. Avoid: Running a single production replica without health checks, exposing app ports directly without a gateway, and shipping secrets as plain config values.

8. Redis and CDN

Use: Redis for hot read cache and CDN for static files, image variants, media content, and public assets. Why: Redis reduces load on the database, and a CDN reduces network distance and server load for media-heavy pages. Avoid: Serving all assets directly from the application origin, caching private data globally, and shipping uncompressed media at large sizes.

9. Monitoring and Observability

Use: Sentry, structured logs, Prometheus, Grafana, and OpenTelemetry tracing. Why: This gives visibility into errors, latency, throughput, CPU and memory usage, and cross-service delay chains. It is essential for debugging production incidents. Avoid: Relying only on frontend error surfaces, ignoring slow endpoints, and shipping without alerts for 5xx errors or latency thresholds.

10. Background Jobs and Queue Processing

Use: RabbitMQ, Redis queues, or a managed queue service for notifications, exports, email jobs, and long-running workflows. Why: Background jobs decouple heavy work from the request path and improve product responsiveness. Avoid: Running time-consuming processing inside the HTTP API request, which blocks the user and creates queueing under load.

11. Authentication and Authorization

Use: JWT or session-based authentication with secure cookie flags (HttpOnly, Secure, SameSite=Strict), role-based access control (RBAC), and Supabase RLS (Row-Level Security) policies enforced at the database level. Why: Supabase RLS ensures that even if application logic is compromised, the database still enforces user data isolation. JWT with secure cookies prevents token theft and XSS attacks. RBAC creates boundaries between user types and prevents privilege escalation. Avoid: Storing authentication state in localStorage, relying only on application-layer authorization checks without database-level enforcement, passing tokens in URL parameters, and using generic admin roles without granular permissions.

12. Input Validation and Sanitization

Use: Server-side validation for all user inputs, HTML entity encoding for form data, and TypeScript strict types to catch unsafe assignments at build time. Use libraries like Zod or Yup for runtime schema validation. Why: Client-side validation is easily bypassed. Server-side validation prevents SQL injection, XSS, and malformed data from reaching the database or being reflected to other users. Avoid: Trusting client-side form validation alone,

inserting user input directly into HTML or queries, and omitting validation on API endpoints.

13. OWASP Top 10 Mitigation

Use: Dependency scanning (npm audit, Snyk), SQL parameter binding (Supabase client does this), secure headers middleware (Next.js Security Headers), and HTTPS enforcement. Why: The top vulnerabilities (injection, broken auth, XSS, CSRF, XXE, broken access control, etc.) are preventable with established practices. Avoid: Ignoring dependency warnings, concatenating user input into SQL strings, serving content over HTTP in production, and missing security headers.

14. Security Headers and Content Security Policy

Use: Implement Content-Security-Policy (CSP), X-Frame-Options: DENY, X-Content-Type-Options: nosniff, Strict-Transport-Security (HSTS), and Referrer-Policy headers via Next.js middleware or server headers. Why: These headers prevent clickjacking, XSS injection, MIME sniffing, and man-in-the-middle attacks. Avoid: Allowing inline scripts and styles, permitting framing by external domains, and missing HSTS preload in production.

15. CORS and CSRF Protection

Use: Restrict CORS origin list to known domains, use SameSite cookies with Strict or Lax setting, and implement CSRF token validation for state-changing requests (POST, PUT, DELETE). Why: CORS prevents cross-origin API requests from malicious sites. SameSite cookies block CSRF attacks. CSRF tokens ensure requests originate from your application. Avoid: Setting CORS to '*', using SameSite=None without Secure flag in production, and allowing state-changing requests without token validation.

16. Rate Limiting and DDoS Mitigation

Use: Implement rate limiting at the API gateway or middleware layer (e.g., redis-based rate limiting, Cloudflare, AWS WAF). Apply per-user or per-IP limits for login, password reset, and sensitive endpoints. Why: Rate limiting prevents brute-force attacks, credential stuffing, abuse, and reduces DDoS impact. Avoid: No rate limiting on login or public endpoints, hardcoded limits that ignore traffic patterns, and missing backoff strategies.

17. Dependency Vulnerability Scanning

Use: Regular npm audit runs, automated dependency updates (Dependabot, Renovate), and security scanning tools (Snyk, npm Audit CI). Why: Third-party packages are regularly discovered to have CVEs. Automated scanning catches vulnerabilities quickly and allows teams to patch before exploitation. Avoid: Ignoring security advisories, running outdated dependencies in production, and skipping CI checks for audit results.

18. Data Encryption in Transit and at Rest

Use: TLS 1.3+ for all connections (HTTPS), encrypted database connections, and encryption at rest for sensitive data in Supabase. Why: Encryption prevents eavesdropping on network traffic and protects data stored in the database from physical access or backup theft. Avoid: HTTP in production, unencrypted database connections, and storing plaintext passwords or API keys.

19. Session Security and Token Management

Use: Short-lived access tokens (5-15 minutes), refresh token rotation, secure token storage (HttpOnly cookies), and token revocation on logout. Why: Short-lived tokens limit the window for token theft exploitation. Refresh token rotation prevents indefinite access after a compromise. HttpOnly prevents JavaScript access to tokens. Avoid: Long-lived tokens, storing tokens in localStorage, no token revocation mechanism, and missing refresh token rotation.

20. API Authentication Strategies

Use: API Key authentication for backend-to-backend calls (service authentication), OAuth 2.0 for third-party integrations, and JWT for user sessions. Use strong key generation and rotation. Why: Multiple authentication strategies provide defense-in-depth and allow fine-grained access control for different client types. Avoid: Hardcoding API keys, sharing authentication between services, and missing key rotation policies.

21. Logging and Security Auditing

Use: Structured logging of authentication events, authorization decisions, and sensitive operations (data export, account changes). Use Sentry for error tracking and a SIEM tool for security event aggregation. Why: Audit logs provide forensic evidence of breaches and help detect unauthorized access patterns. Avoid: Logging sensitive data (passwords, tokens, PII), insufficient logging of auth events, and no centralized log aggregation.

22. Incident Response and Security Updates

Use: A documented incident response plan, automated alerts for security anomalies, and a patching strategy for critical vulnerabilities. Why: Rapid response to incidents and patches minimizes damage and recovery time. Avoid: No incident response process, delayed security patches, and missing communication plan for security events.

23. Secret and Config Management

Use: Kubernetes Secrets, Azure Key Vault, AWS Secrets Manager, or similar secure secret storage. Rotate secrets regularly and use short-lived credentials where possible. Why: This keeps API keys, service credentials, and environment values secure, isolated from code, and manageable across environments. Avoid: Committing secrets in source code, Docker build args, plain environment files in repo-controlled locations, and using static long-lived credentials for service-to-service auth.

24. Database Tuning and Query Health

Use: Proper indexes, query profiling, database metrics, pagination, and read-splitting strategy where needed. Use Supabase built-in monitoring. Why: High-traffic tables will degrade rapidly without proper indexes and query hygiene. Good query health improves performance and resilience. Avoid: Unbounded queries, no indexes on common filters, large sequential scans across production tables, and missing database performance monitoring.

25. Recommended Production Architecture

Use: Next.js frontend tier, server data layer, Redis cache, PostgreSQL/Supabase data layer with RLS policies, background workers, ingress routing, CDN, monitoring with Sentry, security scanning, and container orchestration. Why: This creates a scalable, resilient, secure, and maintainable platform with clear separation between request

handling, data access, background processing, and infrastructure operations. Security is layered at the database, application, and infrastructure levels. Avoid: Building a single giant service without boundaries, adding microservices before core bottlenecks are known, or deferring security hardening until after launch.

26. Security Implementation Roadmap

Phase 1: Enable Supabase RLS, add input validation, implement secure cookies and HTTPS. Phase 2: Add security headers middleware, CORS restrictions, rate limiting, and CSRF tokens. Phase 3: Implement audit logging, secrets management, dependency scanning, and incident response plan. Phase 4: Add encryption at rest, API key rotation, and advanced threat monitoring. Why: Phased implementation reduces risk, allows team learning, and prioritizes high-impact, low-effort security wins first. Avoid: Trying to implement all security measures at once, deferring security to the end, and launching production without Phase 1 controls.

27. Final Recommendation

Use: A layered, phased implementation plan that starts with frontend efficiency, server-side optimization, caching, and external API hardening, then moves into security hardening (auth, input validation, headers, rate limiting), and finally orchestration and autoscaling. Why: This produces the highest return on effort, reduces production risk, improves security posture incrementally, and preserves app quality while addressing real bottlenecks. Avoid: Jumping straight to large-scale orchestrated architecture before fixing bottlenecks, deferring security until after launch, and implementing advanced features without foundational security controls.

Prepared for: ConnectIQO Web Portal